



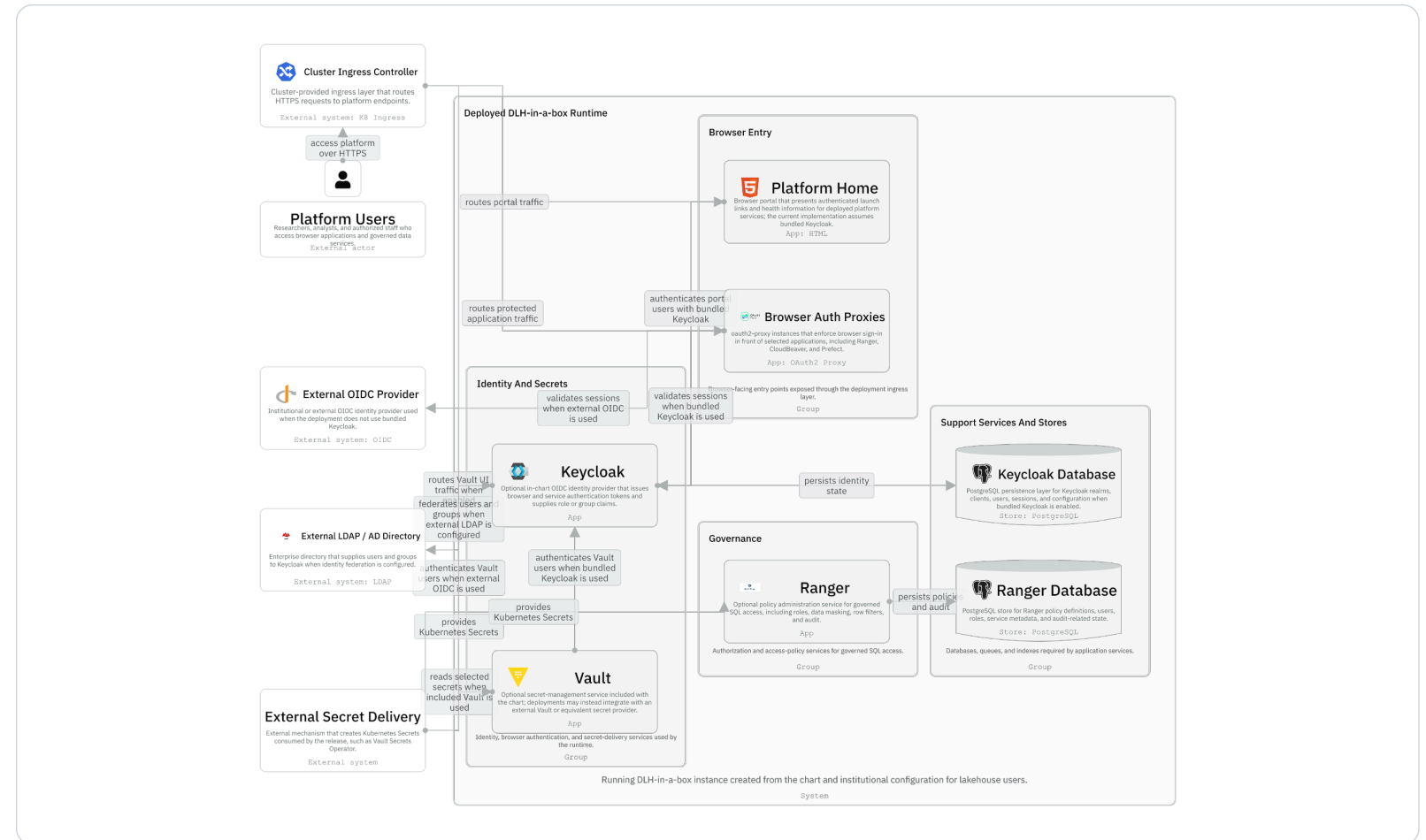
Session 05 / Auth modes and entryptoints

Identity And Browser Access

One hour handover deck

40-45 min narrative / 10-15 min checkpoint /
5 min Q&A

The chart orchestrates a system-wide identity contract rather than configuring each browser app independently.



Session path

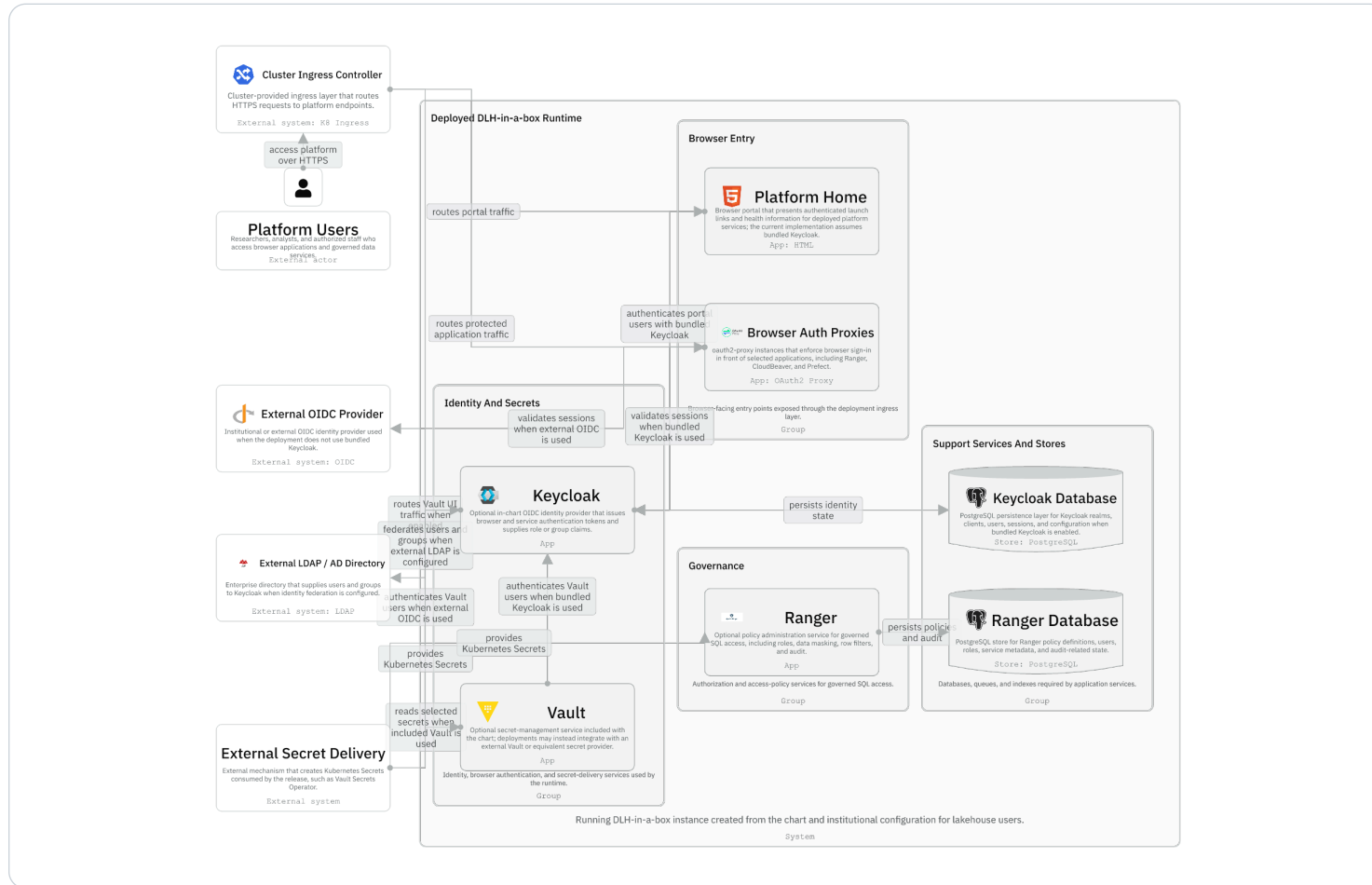
- 1 **Two identity axes**
- 2 **Browser entrypoints**
- 3 **Direct OIDC versus proxy**
- 4 **Local auth-heavy story**

Session rhythm

40-45 min narrative
10-15 min checkpoint
5 min Q&A

Presenter notes carry the script and demo prompts.

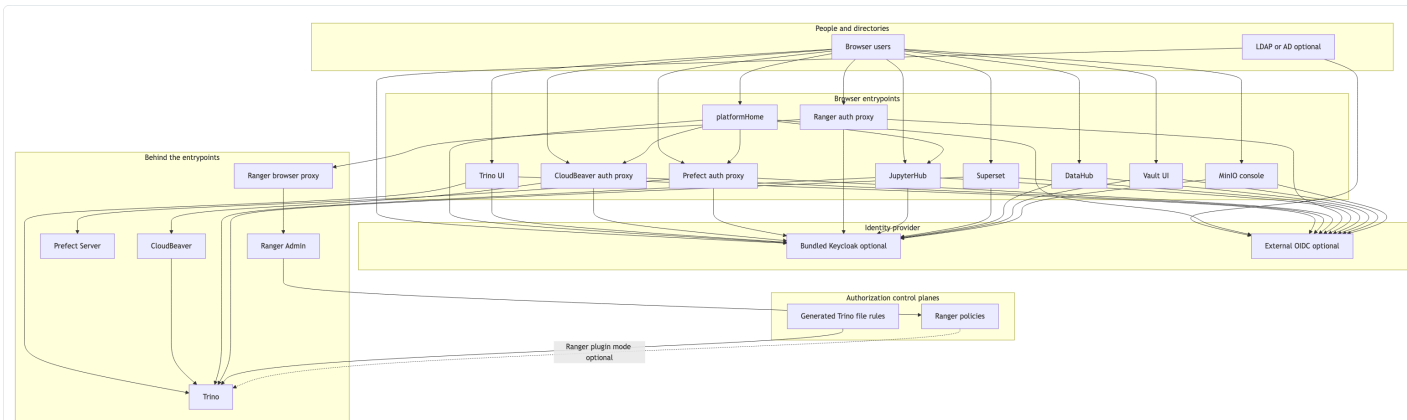
Runtime identity, access, and secrets



- Keycloak or external OIDC issues browser identity.
- Secrets may be supplied externally or by included services.
- Browser auth proxies protect selected apps.
- Trino, Ranger, and app runtimes consume identity differently.

Auth and control flow

- Users enter through browser-facing applications.
- Apps either handle OIDC directly or sit behind oauth2-proxy.
- Ranger policies and generated file rules are authorization control planes.



Identity has two independent axes

Axis	Setting	Meaning
Provider	bundledKeycloak	The chart deploys Keycloak and manages clients
Provider	externalOidc	An existing OIDC provider is consumed
Directory	externalLdap	Users and groups come from LDAP or AD
Directory	keycloakLocal	Keycloak manages local demo users

Supported combinations that matter

Provider	Directory	Use case
bundledKeycloak	externalLdap	Default shared dev and prod pattern
bundledKeycloak	keycloakLocal	Local auth-heavy smoke and demo pattern
externalOidc	externalLdap	Escape hatch when an external IdP exists

Which apps use which browser boundary?

App	Pattern	Why
Trino UI	Direct OIDC	Trino handles login itself
JupyterHub, Superset, DataHub	Direct OIDC	Each app has its own OIDC client wiring
CloudBeaver	oauth2-proxy	Proxy owns browser sign-in
Prefect	oauth2-proxy	Proxy protects browser UI
Ranger browser access	oauth2-proxy plus browser proxy	Ranger Admin is not exposed directly as the main surface

platformHome is special

Current implementation constraint

- It is a launchpad UI and helper API rendered by templates/platform-home.yaml.
- It currently uses the Keycloak JavaScript adapter directly.
- That means platformHome requires bundled Keycloak today.
- Most code lives inline in the template rather than files/platform-home/.

**It is a
launchpad UI
and helper
API rendered
by
templates/platform-home.yaml.**

The local auth-heavy story is intentionally different

Smoke path identity shape

- Bundled Keycloak is enabled.
- Keycloak manages local users directly.
- Ranger LDAP usersync is disabled.
- Demo Secrets are seeded by smoke-install.sh.

**Bundled
Keycloak is
enabled.**

Guided checkpoint: classify an auth problem

Ask: provider mode or directory mode?

Ask: direct OIDC app or oauth2-proxy app?

Ask: browser login problem or backend authorization problem?

Then inspect values, validation output, generated client settings, and relevant app template.

Checkpoint questions

- Can the developer place CloudBeaver, Trino UI, and platformHome on the right boundary?
- Can they explain why a local auth smoke issue may involve seeded Secrets?

Session 05 landing

What the inheriting team should now be able to do

- They can separate provider mode from directory mode.
- They know which apps use direct OIDC versus proxy boundaries.
- They understand the platformHome bundled-Keycloak constraint.
- They can triage auth issues by boundary instead of by product name alone.

Leave-behind

Use the PPTX notes as a presenter script and source-map.md as the reference trail.